Introdução JavaServer Pages Standard Tag Library

Conteudista: Prof. Me. Alexander Gobbato Paulino Albuquerque
Revisão Textual: Prof. Claudio Brites

Objetivos da Unidade:

- Aprofundar no desenvolvimento de aplicações *web* utilizando as bibliotecas do JSTL;
- Adquirir conhecimento para a criação de um projeto utilizando html e JSTL.

Material Teórico

Material Complementar

Referências

Material Teórico

Introdução *JavaServer Pages Standard Tag Library*

Nesta Unidade, já estamos com o ambiente preparado e já vimos como criar uma aplicação em JSP.

Desse ponto em diante, utilizaremos a diretiva de *tags* da biblioteca *JSTL* (*JavaServer Pages Standard Tag*), mas o que é e para que servem essas bibliotecas?

Essa biblioteca de *tags* é uma coleção de *tags* JSP que tem por função principal "ajudar" em elementos em *java* que serão colocados juntamente com o html. Dessa forma, não iremos "misturar" o *java* com hmtl.

O que é necessário para utilizarmos essas bibliotecas? Primeiramente, precisamos fazer um *download* dessas bibliotecas, após o *download*, precisamos colocar os arquivos .jar que foram baixados no diretório **webapps\ROOT\WEB-INF\lib**. Agora, quando formos usar dentro das páginas .jsp, precisamos incluir uma *taglib* (diretiva) para que possamos ter acesso.

Como utilizar a biblioteca:

- **JSTL *Core Tags*:** Essas *tags* são usadas para iteração, verificação de condição, gerenciamento de URL, entre outras funcionalidades. São as *tags* principais mais usadas.

  Sintaxe: <%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>;

- **JSTL *Formatting Tags*:** Essas *tags* são usadas para formatar o texto, data, números etc.

  Sintaxe: <%@ taglib uri="http://java.sun.com/jsp/jstl/fmt" prefix="fmt"%>;

- **SQL *Tags*:** Essas *tags* são usadas para interagir com bancos de dados como Oracle, MySQL etc.

  Sintaxe: <%@ taglib uri="http://java.sun.com/jsp/jstl/sql" prefix="sql"%>;

- **JSTL *Functions*:** Essas *tags* de função são usadas principalmente para manipular *strings*.

  Sintaxe: <%@ taglib uri="http://java.sun.com/jsp/jstl/functions" prefix="fn" %>.

Veja, a seguir, como baixar as bibliotecas para o seu projeto:

Site

Tomcat

Acesse o *link* a seguir e clique em *Apache Stardard Taglib*.

Clique no botão para conferir o conteúdo.

ACESSE

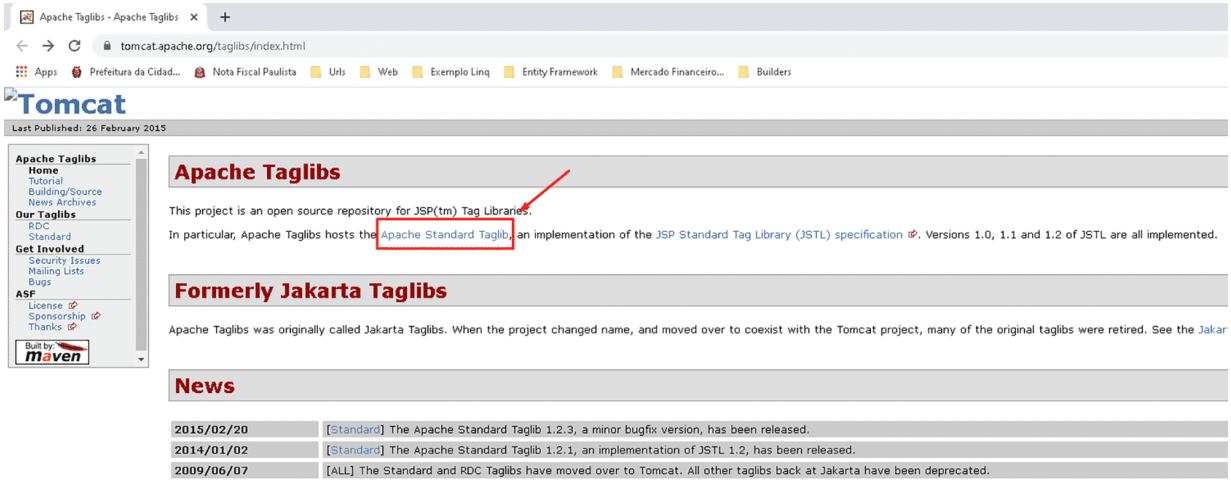


Figura 1
Fonte: Reprodução

A seguir clique na opção *download* da versão JSTL 1.2.

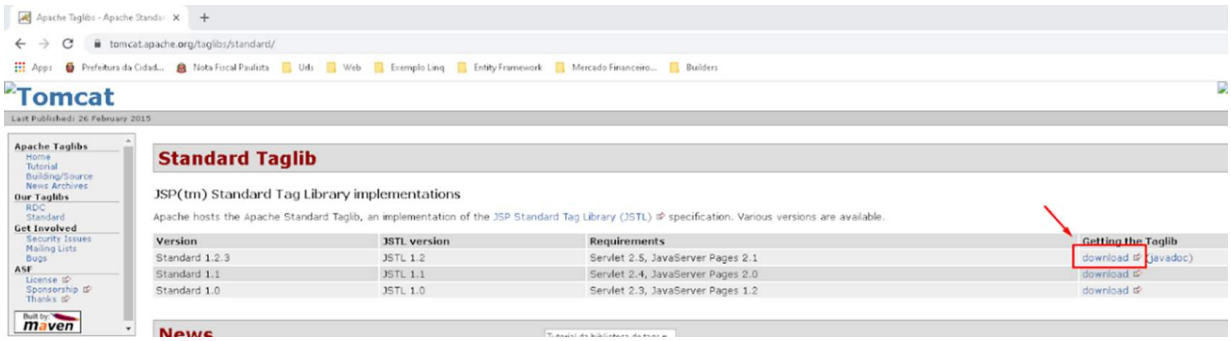


Figura 2
Fonte: Reprodução

Baixar os arquivos .jar.

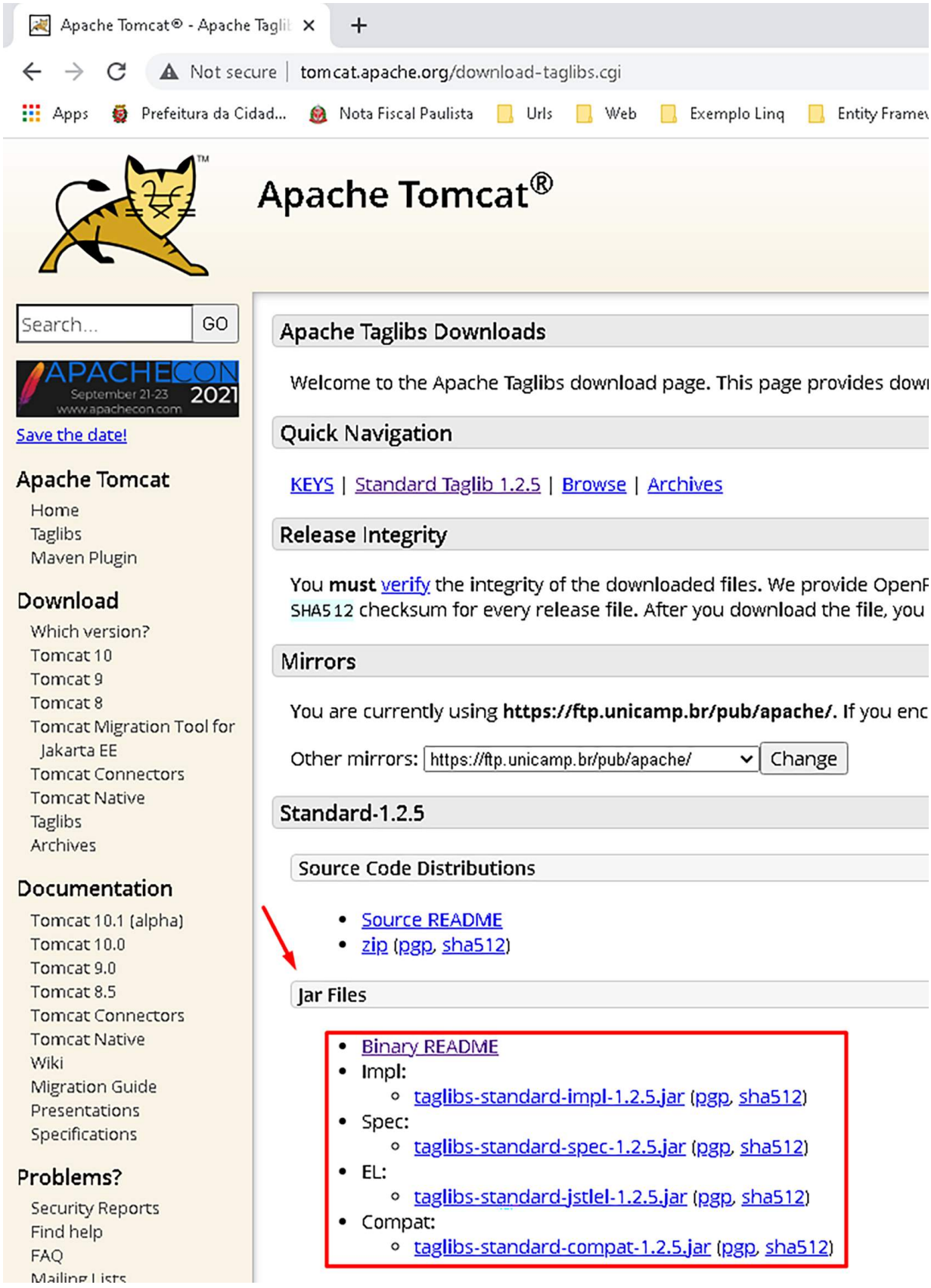

Apache Tomcat® - Apache Taglil
Not secure | tomcat.apache.org/download-taglibs.cgi
Apps
Prefeitura da Cidad...
Nota Fiscal Paulista
Urls
Web
Exemplo Linq
Entity Framev
Apache Tomcat®
Search...
GO
APACHECON
September 21-23
2021
www.apachecon.com
Save the date!
Apache Tomcat
Home
Taglibs
Maven Plugin
Download
Which version?
Tomcat 10
Tomcat 9
Tomcat 8
Tomcat Migration Tool for Jakarta EE
Tomcat Connectors
Tomcat Native
Taglibs
Archives
Documentation
Tomcat 10.1 (alpha)
Tomcat 10.0
Tomcat 9.0
Tomcat 8.5
Tomcat Connectors
Tomcat Native
Wiki
Migration Guide
Presentations
Specifications
Problems?
Security Reports
Find help
FAQ
Mailing Lists
Apache Taglibs Downloads
Welcome to the Apache Taglibs download page. This page provides dowr
Quick Navigation
KEYS | Standard Taglib 1.2.5 | Browse | Archives
Release Integrity
You must verify the integrity of the downloaded files. We provide OpenF
SHA512 checksum for every release file. After you download the file, you
Mirrors
You are currently using https://ftp.unicamp.br/pub/apache/. If you enc
Other mirrors: https://ftp.unicamp.br/pub/apache/ Change
Standard-1.2.5
Source Code Distributions
Source README
zip (pgp, sha512)
Jar Files
Binary README
Impl:
taglibs-standard-impl-1.2.5.jar (pgp, sha512)
Spec:
taglibs-standard-spec-1.2.5.jar (pgp, sha512)
EL:
taglibs-standard-jstlel-1.2.5.jar (pgp, sha512)
Compat:
taglibs-standard-compat-1.2.5.jar (pgp, sha512)

Figura 3
Fonte: Reprodução

E no projeto do *Eclipse*, copiar para a pasta lib, como exibido na Figura a seguir:

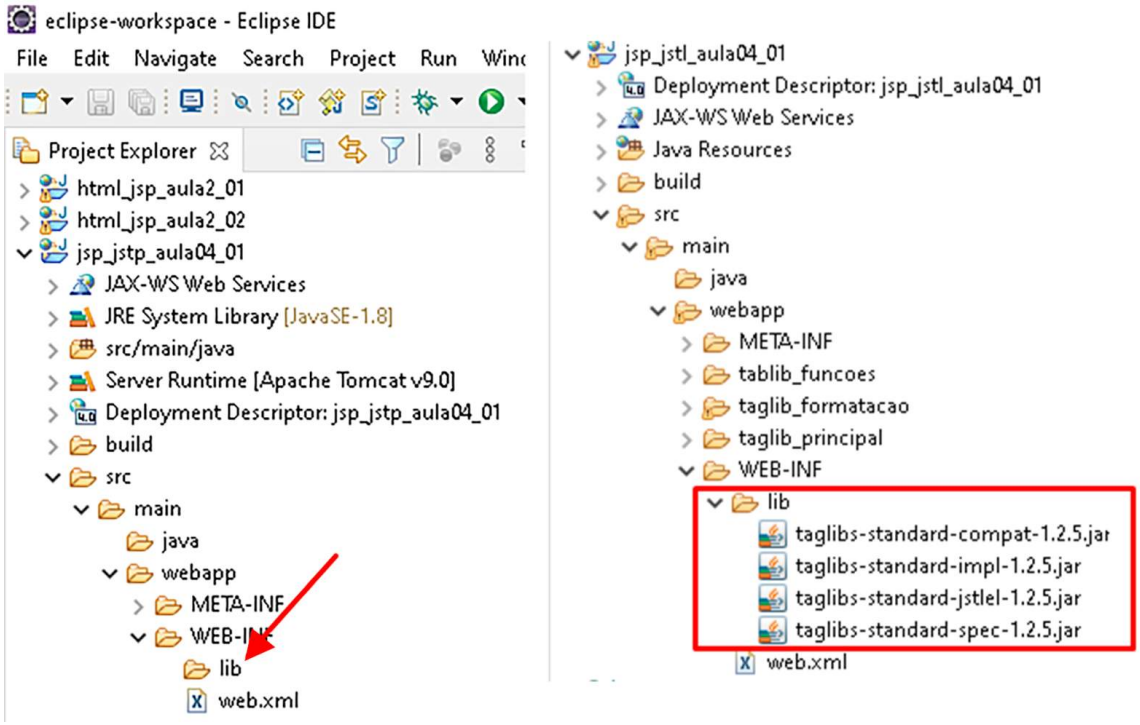


Figura 4
Fonte: Reprodução

Agora que sabemos como configurar a ferramenta para utilizar os elementos do JSTL, vamos colocar a mão na massa.

Precisamos criar um projeto com o tipo *Dynamic Web Project* e então preencher algumas informações como nome do projeto, Servidor (*Apache Tomcat*), Selecionar a pasta dos arquivos e, por último, selecionar o *Generate* web.xml para que a IDE crie as informações de configurações do projeto. Todos esses passos estão mapeados nas Figuras a seguir.

Não esqueça de incluir os arquivos da biblioteca JSTL que foram baixados do *site* e copiar para a pasta lib do projeto criado.

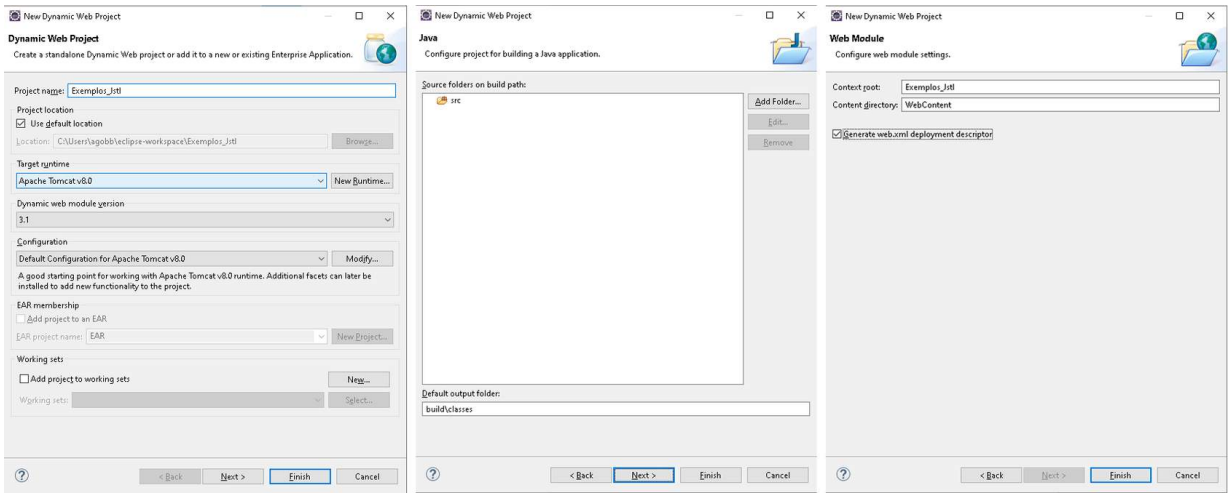


Figura 5
Fonte: Reprodução

Com o projeto criado, agora precisamos criar uma página JSP e na página fazer uma referência a *taglib*. A página deve ser criada dentro do *webapp* ou *WebContent*. Isso vai depender da versão do seu eclipse, mas ambas as pastas têm a mesma finalidade.

Com o arquivo Exemplo_Jstl.jsp criado, é exibido o código a seguir.

Exemplo_Jstl.jsp

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
</head>
<body>

</body>
</html>
```

Figura 6
Fonte: Reprodução

Agora vamos colocar em prática o que foi falado anteriormente e utilizar o JSTL.

Primeiro devemos incluir a linha que fará referência às bibliotecas.

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>
```

Observe que a *taglib* tem um endereço na uri. O endereço em questão é para utilizar as principais *tags* dessa biblioteca, outro detalhe, e até mesmo um padrão utilizado no mercado de trabalho, é utilizar um nome no atributo *prefix*, como nesse exemplo. Todas as funções estão sendo representadas pela letra 'c'.

Uma das funções existentes é o *set*, ela é utilizada para que um valor possa ser definido, fixo ou dinamicamente, já o *out* tem a finalidade de "imprimir na tela" uma determinada informação.

Veja o exemplo:

```jsp
<%@ page language="java" contentType="text/html; charset=ISO-8859-1" pageEncoding="ISO-8859-1"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title></title>
</head>
<body>
    <c:set var="salary" scope="session" value="${2000*2}" />
    <c:out value="${salary}" />
</body>
</html>
```

Figura 7
Fonte: Reprodução

Podemos também testar algumas condições, ou seja, utilizar a estrutura condicional para verificar algumas informações e direcionar o programa para algum ponto do código.

Veja o exemplo:

*Exemplo_Jstl.jsp Utilizando o JSTL

```jsp
<%@ page language="java" contentType="text/html; charset=ISO-8859-1" pageEncoding="ISO-8859-1"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Utilizando o JSTL</title>
</head>
<body>
    <c:set var="salary" scope="session" value="${2000*2}" />
    O salário é: <c:out value="${salary}" />
    <c:if test="${salary >= 2000}">
        <p>
            <c:out value="${'Salário maior ou igual a 2000'}" />
        <p>
    </c:if>
    <c:if test="${salary < 2000}">
        <p>
            <c:out value="${'Salário menor do que 2000'}" />
        <p>
    </c:if>
</body>
</html>
```

Figura 8
Fonte: Reprodução

Resultado da execução:

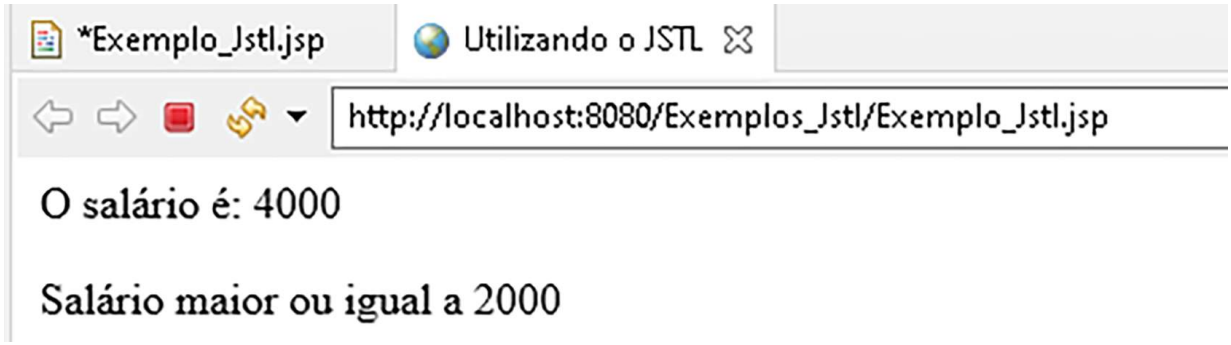


Figura 9
Fonte: Reprodução

As funções do JSTL não possuem o *else*, para isso podemos utilizar a função *choose*. Nessa função, veja que as condições são testadas com a cláusula *When*, e se nenhuma das condições forem atendidas, executa a instrução do código que está em *otherwise*.

Exemplo_Jstl.jsp

```jsp
<%@ page language="java" contentType="text/html; charset=ISO-8859-1" pageEncoding="ISO-8859-1"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Utilizando o JSTL</title>
</head>
<body>
    <c:set var="salary" scope="session" value="${2000*2}" />
    O salário é: <c:out value="${salary}" />
    <c:choose>
        <c:when test="${salary <= 0}">
            Salário muito baixo para sobreviver.
         </c:when>

        <c:when test="${salary > 1000}">
            Salário está bom.
         </c:when>

        <c:otherwise>
            Qualquer outra condição que não foi atendida anteriormente.
         </c:otherwise>
    </c:choose>
</body>
</html>
```

Figura 10
Fonte: Reprodução

Vamos entender agora algumas funções de formatação de datas e números.

A biblioteca do JSTL vem separada por funcionalidades. Anteriormente utilizamos a padrão, também conhecida por *core*, nesse caso, como trabalharemos com números e datas, utilizaremos a fmt, então para os exemplos a seguir devemos acrescentar a seguinte *taglib* ao arquivo .jsp.

```
<%@ taglib prefix="fmt" uri="http://java.sun.com/jsp/jstl/fmt"%>
```

Vale fazer uma ressalva aqui, caso seja necessário podemos utilizar a *core* e fmt no mesmo arquivo.

Para formatar uma data que será "impressa" na tela, existem alguns tipos que podemos utilizar, por exemplo, ***type="time"*** ou ***type="date"*** ou ***type="both"***. Podemos também configurar a forma em que a data será exibida utilizando o *pattern*.

Exemplo:

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1" pageEncoding="ISO-8859-1"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>
<%@ taglib prefix="fmt" uri="http://java.sun.com/jsp/jstl/fmt"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Utilizando o JSTL</title>
</head>
<body>
    <c:set var="now" value="<%=new java.util.Date()%>" />
    <c:out value="${now}" />
    <p>
        Formatando Hora:
        <fmt:formatDate type="time" value="${now}" />
    </p>
    <p>
        Formatando Data:
        <fmt:formatDate type="date" value="${now}" />
    </p>
    <p>
        Formatando Data e Hora:
        <fmt:formatDate type="both" value="${now}" />
    </p>
    <p>
        Formatando Data para Dia, Mês, Ano:
        <fmt:formatDate pattern="dd-MM-yyyy" value="${now}" />
    </p>
</body>
</html>
```

Figura 11
Fonte: Reprodução

Resultado:

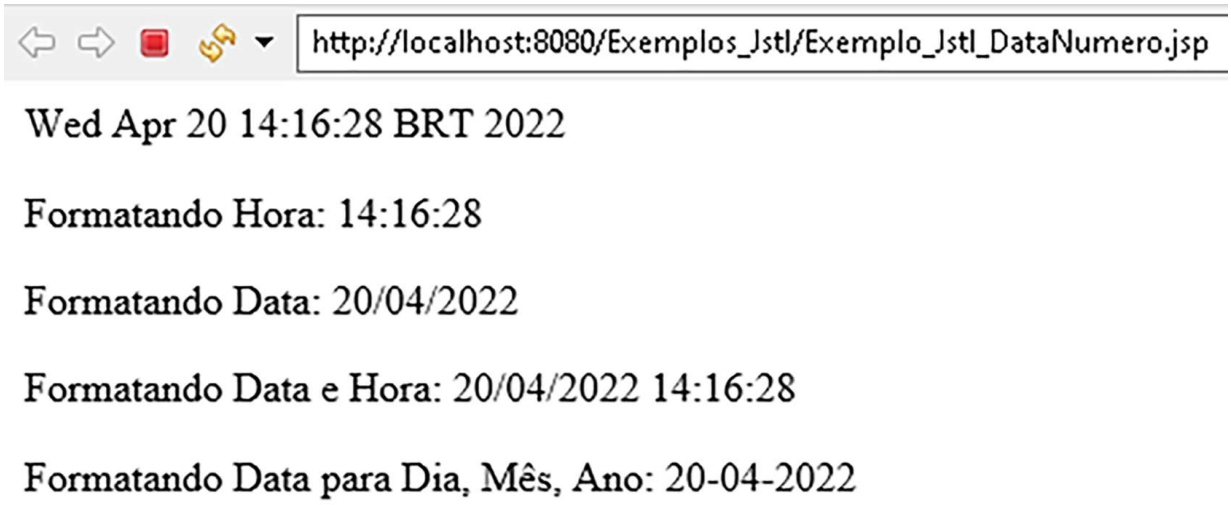


Figura 12
Fonte: Reprodução

Já na formatação do número, podemos trabalhar com o tipo moeda, casas decimais, porcentagens e moeda local.

Exemplo:

```jsp
<%@ page language="java" contentType="text/html; charset=ISO-8859-1" pageEncoding="ISO-8859-1"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>
<%@ taglib prefix="fmt" uri="http://java.sun.com/jsp/jstl/fmt"%>
<!DOCTYPE html>
<html>
<head>
</head>
<body>
    <c:set var="balance" value="120000.2309" />
    <c:set var="porc" value="10" />
    <p>
        Moeda local:
        <fmt:formatNumber value="${balance}" type="currency" />
    </p>
    <p>
        Porcentagem:
        <fmt:formatNumber type="percent" groupingUsed="true" value="${porc}" />
    </p>
    <p>
        Formatada com notação exponencial:
        <fmt:formatNumber type="number" pattern="###.###E0" value="${balance}" />
    </p>
    <p>
        Moeda local americana :
        <fmt:setLocale value="en_US" />
        <fmt:formatNumber value="${balance}" type="currency" />
    </p>
</body>
</html>
```

Figura 13
Fonte: Reprodução

Resultado:

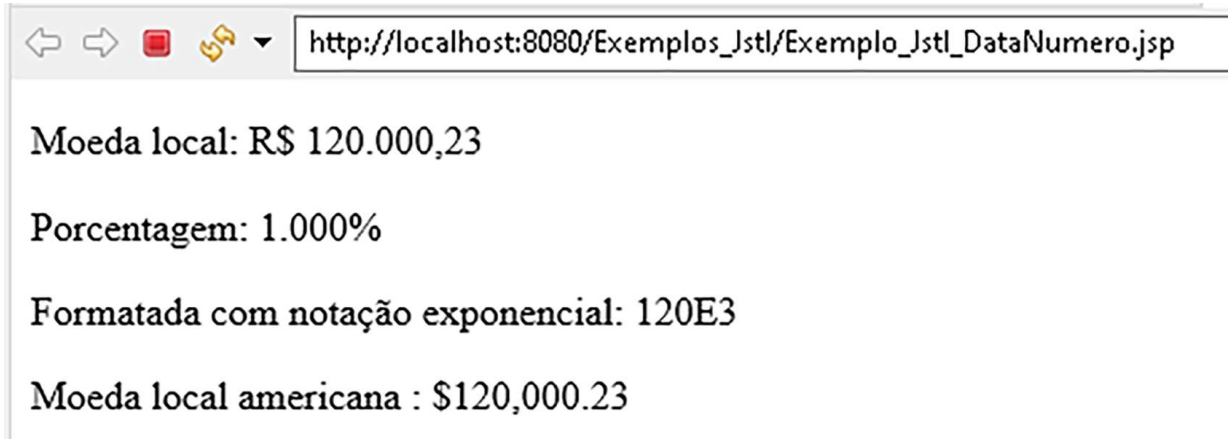


Figura 14
Fonte: Reprodução

Com as bibliotecas do JSTL, podemos realizar várias operações, podemos inclusive trabalhar com XML e Banco de Dados.

No exemplo a seguir, será exibido como criar uma conexão utilizando o *driver* do *MySQL* e, ao se conectar no banco de dados, realizaremos uma pesquisa no banco, onde serão retornados alguns registros. Esses registros serão trabalhos com a *tag Core* para fazer uma iteração de repetição e "printar" na tela.

Como utilizaremos o banco de dados, precisamos trabalhar com a *taglib* específica para essa demanda.

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/sql" prefix="sql"%>
```

Para conectar no banco, precisaremos utilizar a função do *setDataSource*, onde informaremos ao *driver* do *MySQL* o caminho do servidor *MySQL*, usuário e senha.

A instrução *query* irá executar o *select* e seu o resultado será armazenado na variável *result*.

Nessa variável, ficará armazenado o resultado do banco, e, então, através do *forEach*, serão percorridos todos os registros.

Veja o código a seguir e o seu resultado em html.

Bloco com as *taglibs* utilizadas:

```
1 <%@ page language="java" contentType="text/html; charset=ISO-8859-1"
2     pageEncoding="ISO-8859-1"%>
3 <%@ page import="java.io.*,java.util.*,java.sql.*"%>
4 <%@ page import="javax.servlet.http.*,javax.servlet.*"%>
5 <%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>
6 <%@ taglib uri="http://java.sun.com/jsp/jstl/sql" prefix="sql"%>
```

Figura 15

Código para realizar a conexão com o banco e realizar o *select* na tabela contato:

```
14    <sql:setDataSource var="snapshot" driver="com.mysql.jdbc.Driver"
15         url="jdbc:mysql://localhost:3306/agenda?useSSL=false&serverTimezone=UTC"
16         user="root"  password="root"/>
17    <sql:query dataSource="${snapshot}" var="result">
18        select * from agenda.contato
19      </sql:query>
```

Figura 16

Como o resultado da operação ficará armazenado na variável *result*, precisamos realizar um laço para ler registro a registro retornado.

```
21    <table border="1" width="100%">
22        <tr>
23            <th>Emp ID</th>
24            <th>Nome</th>
25            <th>E-mail</th>
26        </tr>
27
28        <c:forEach var="row" items="${result.rows}">
29            <tr>
30                <td><c:out value="${row.Id}" /></td>
31                <td><c:out value="${row.nome}" /></td>
32                <td><c:out value="${row.email}" /></td>
33            </tr>
34        </c:forEach>
35    </table>
```

Figura 17

Ao executar o projeto, temos o seguinte resultado:



Figura 18

Entendendo o código:

Primeiro para trabalharmos com como as bibliotecas precisamos importá-las.

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>

<%@ taglib uri="http://java.sun.com/jsp/jstl/sql" prefix="sql"%>
```

Como estamos fazendo um acesso ao banco de dados *MySQL*, precisamos informar qual usuário, senha e banco. Para desenvolvermos essa funcionalidade, precisamos do recurso *setDataSource*, em que informamos o *driver* do banco e em qual tabela faremos a pesquisa.

Depois disso, o resultado da pesquisa é armazenado na variável *result*, em que percorremos registro a registro para então exibirmos o resultado na tela!

Para finalizar, o uso da biblioteca JSTL é de grande utilidade, já que a combinação de *java*, jsp e hmtl está “embutido” na biblioteca.

Material Complementar

Indicações para saber mais sobre os assuntos abordados nesta Unidade:

Leitura

JSTL *core* – *Oracle Docs*

Clique no botão para conferir o conteúdo.

ACESSE

JSTL fmt – *Oracle Docs*

Clique no botão para conferir o conteúdo.

ACESSE

JSTL sql – *Oracle Docs*

Clique no botão para conferir o conteúdo.

ACESSE

JSTL *functions – Oracle Docs*

Clique no botão para conferir o conteúdo.

ACESSE

Referências

JAVASERVER Pages Standard Tag Library 1.1 *Tag Reference*. ***Oracle Docs***, s. d. Disponível em: <https://docs.oracle.com/javaee/5/jstl/1.1/docs/tlddocs/>. Acesso em 03/06/2022.